Module: React Native for Beginners – Build Your First Mobile App

Nov 9, 2026 · @albert

Module overview

This module takes students from zero to a working two-screen React Native app, following the chapters of the *React Native Tutorial for Beginners – Build a React Native App* video by Programming with Mosh. It is built from the video's published chapter list, so check the lesson details against the video before class.

Item	Detail
Fits	IT and web development courses (for example ADET)
Suggested time	3 sessions of about 2 hours each
Prerequisites	JavaScript basics; React basics (components, props, state, JSX)
Tools	Node.js, a code editor (VS Code), Expo, a phone with Expo Go or an emulator
Source video	React Native Tutorial for Beginners – Build a React Native App
Final output	A Welcome screen and a View Image screen, refactored into reusable pieces

How to use it. Students watch the matching video chapters, read the lesson, then code along. Each lesson ends with a short check; the hands-on project and the quiz come last.

Learning objectives

After this module, students can build and run a small cross-platform mobile app on their own. Specifically, they can:

1. Explain what React Native is and how it differs from a web app wrapped in a mobile shell.

2. Explain why Expo is the easiest way to start, and create and run a project with it.
3. Run an app on an iOS simulator, an Android emulator or a physical device, and debug it using logs and a debugger.
4. Use the core components: View, Text, Image, Touchables, Button and Alert.
5. Style components with StyleSheet and write platform-specific code.
6. Build responsive layouts with Dimensions, orientation detection and Flexbox, including absolute and relative positioning.
7. Combine these skills into a Welcome screen and a View Image screen, then refactor the code into reusable components.

Lesson 1: What is React Native, and setting up

Video chapters: Introduction, Prerequisites, What is React Native?, Expo, Setting up the development environment.

React Native lets you build real iOS and Android apps with JavaScript and React. Your components render as native UI elements, not as web pages inside a mobile shell, so one codebase gives you apps that look and feel native on both platforms.

Expo or React Native CLI? Expo is a framework built on React Native. It handles the native build setup so beginners can run a first app in minutes, without installing Xcode or Android Studio. The React Native documentation itself recommends starting with a framework such as Expo. The plain React Native CLI gives more control but needs the full native toolchain.

	Expo	React Native CLI
Setup effort	Low: Node.js plus one command	High: Xcode, Android Studio, SDKs
Test on a phone	Scan a QR code with Expo Go	Build and install the app
Native code control	Limited at first, extendable later	Full
Best for	Learning, prototypes, most apps	Apps needing custom native modules

Setting up. Install Node.js and VS Code, then create and start a project:

```
npx create-expo-app@latest my-app
cd my-app
npx expo start
```

The video may use an older Expo toolchain, so its setup commands can differ from these. Follow the current Expo guide when they disagree. If you test on a physical phone with Expo Go, make sure the project's SDK version matches what your Expo Go supports.

Check yourself

- Name two reasons a beginner would choose Expo.
- Why is a React Native app not just a website in a mobile wrapper?

Lesson 2: Your first app, running it and debugging it

Video chapters: Your First React Native App, Running on an iOS simulator, Running on an Android emulator, Running on a physical device, Logging, Debugging with Chrome, Debugging in VSCode, Publishing.

A React Native app is a React component tree. The smallest useful app is one component that returns a View containing a Text:

```
import { Text, View } from 'react-native';

export default function App() {
  return (
    <View style={{ flex: 1, justifyContent: 'center', alignItems: 'center'
}}>
      <Text>Hello, React Native!</Text>
    </View>
  );
}
```

Save the file and the running app reloads on its own. This fast feedback loop is the main reason to code along live in class.

Three ways to run the app

Target	What you need	Good for
iOS simulator	A Mac with Xcode	Testing iPhone layouts

Target	What you need	Good for
Android emulator	Android Studio and a virtual device	Testing Android layouts on any computer
Physical device	Expo Go app; phone and computer on the same network	Real touch, real speed

Logging and debugging. `console.log()` output appears in the terminal running Expo. For breakpoints and inspecting state, use a debugger. The video shows Chrome and VS Code debugging. Newer React Native versions ship their own developer tools, so check the current React Native debugging docs for the exact steps.

Publishing. Expo apps are built into store-ready iOS and Android binaries through Expo's cloud build service (EAS). Students do not need to publish in class, but should understand that the same project produces both apps.

Check yourself

- Where does `console.log` output appear in an Expo project?
- Which run target is best for judging real touch performance, and why?

Lesson 3: Core components

Video chapters: Fundamental Concepts, View, Text, Image, Touchables, Button, Alert.

React Native has no HTML tags. Instead you import components from `react-native`, and each one maps to a native UI element.

Component	Role	Key points
`View`	Container, like a `div`	Holds and lays out other components
`Text`	Displays text	All text must sit inside a `Text`
`Image`	Displays pictures	Local images use `require`; network images use `{ uri }` and need an explicit width and height
Touchables	Make anything tappable	`TouchableOpacity` fades on press; `TouchableHighlight` shows an underlay; `TouchableWithoutFeedback` gives no visual cue
`Button`	Basic platform button	Takes `title` and `onPress`; limited styling

Component	Role	Key points
`Alert`	Native pop-up dialog	`Alert.alert(title, message, buttons)`

Example: a tappable image with an alert

```
import { Alert, Image, Text, TouchableOpacity, View, Button } from 'react-
native';

export default function App() {
  return (
    <View style={{ flex: 1, justifyContent: 'center', alignItems: 'center'
}}>
      <Text numberOfLines={1} onPress={() => console.log('Text tapped')}>
        Tap me
      </Text>
      <TouchableOpacity onPress={() => console.log('Image tapped')}>
        <Image
          source={{ uri: 'https://picsum.photos/200/300', width: 200, height:
300 }}
        />
      </TouchableOpacity>
      <Button
        title="Click me"
        onPress={() =>
          Alert.alert('My title', 'My message', [
            { text: 'Yes', onPress: () => console.log('Yes') },
            { text: 'No' },
          ])
        }
      />
    </View>
  );
}
```

The React Native docs also offer `Pressable`, a newer and more flexible alternative to the Touchables, which students will meet in other tutorials.

Check yourself

- Why does a network image need a width and height while a local image often does not?
- When would you choose a Touchable over a Button?

Lesson 4: Styling and platform-specific code

Video chapters: StyleSheet, Platform-specific code.

React Native styles are JavaScript objects, not CSS files. Property names use camelCase (`backgroundColor`, not `background-color`), and there are no units: numbers are density-independent pixels.

`StyleSheet.create` groups styles in one place, keeps the JSX readable and lets you reuse a style across components:

```
import { StyleSheet, Text, View } from 'react-native';

export default function Card() {
  return (
    <View style={styles.card}>
      <Text style={styles.title}>Hello</Text>
    </View>
  );
}

const styles = StyleSheet.create({
  card: { backgroundColor: 'dodgerblue', padding: 20, borderRadius: 8 },
  title: { color: 'white', fontSize: 24, fontWeight: '700' },
});
```

Styles are not inherited the way CSS cascades. A `Text` does not pick up a font size from its parent `View`, so set text styles on the `Text` itself.

Platform-specific code. iOS and Android differ in fonts, shadows and conventions. Use the `Platform` API to branch by operating system:

```
import { Platform, StyleSheet } from 'react-native';

const styles = StyleSheet.create({
  title: {
    fontSize: 20,
    fontFamily: Platform.OS === 'android' ? 'Roboto' : 'Avenir',
  },
  box: {
    ...Platform.select({
      ios: { backgroundColor: 'tomato' },
      android: { backgroundColor: 'gold' },
    }),
  },
});
```

For larger differences, create two files named `Component.ios.js` and `Component.android.js`. React Native loads the right one automatically.

Check yourself

- Rewrite `border-radius: 8px` as a React Native style.
- Give one reason to use `Platform.select` instead of two separate component files.

Lesson 5: Layouts, Flexbox and positioning

Video chapters: Layouts, Dimensions, Detecting orientation changes, Flexbox, flexDirection, justifyContent / alignItems / alignSelf, flexWrap / alignContent, flexBasis / flexGrow / flexShrink, Absolute and Relative Positioning.

Every React Native View is a flex container. The key difference from the web is that the default `flexDirection` is `column`, so children stack top to bottom.

Screen size and orientation. Read the screen size with the built-in hook, which re-renders when the device rotates:

```
import { useWindowDimensions } from 'react-native';

const { width, height } = useWindowDimensions();
const isLandscape = width > height;
```

The older `Dimensions.get('window')` returns the size once, so it does not update on rotation. To allow rotation in an Expo app, set `"orientation": "default"` in `app.json`.

Flexbox properties

Property	Controls	Values to know
`flex`	How much space a child takes	`1` fills available space
`flexDirection`	Main axis	`column` (default), `row`, `column-reverse`, `row-reverse`
`justifyContent`	Alignment along the main axis	`flex-start`, `center`, `flex-end`, `space-between`, `space-around`, `space-evenly`
`alignItems`	Alignment along the cross axis	`stretch` (default), `flex-start`, `center`, `flex-end`
`alignSelf`	One child overriding `alignItems`	Same values as `alignItems`
`flexWrap`	Whether items wrap to a new line	`nowrap` (default), `wrap`
`alignContent`	Spacing between wrapped lines	Same values as `justifyContent`; only works with `flexWrap: 'wrap'`
`flexBasis`	Starting size before growing or shrinking	A number or percentage
`flexGrow` / `flexShrink`	How a child grows or shrinks to fit	Numbers; `0` stops it

Example: three boxes in a row, centered

```
<View style={{ flex: 1, flexDirection: 'row', justifyContent: 'center',
alignItems: 'center' }}>
  <View style={{ width: 80, height: 80, backgroundColor: 'dodgerblue' }} />
  <View style={{ width: 80, height: 80, backgroundColor: 'tomato' }} />
  <View style={{ width: 80, height: 80, backgroundColor: 'gold' }} />
</View>
```

Positioning. Elements are `relative` by default: `top`, `left`, `right` and `bottom` nudge them from their normal spot. With `position: 'absolute'` an element leaves the normal flow and is placed against its parent's edges. Use absolute positioning for overlays such as

a close icon pinned to a corner, and Flexbox for everything else.

Check yourself

- Which property would you change to put children side by side?
- What is the difference between `justifyContent` and `alignItems`?
- Why does `alignContent` have no effect without `flexWrap`?

Lesson 6: Hands-on project — Welcome screen, View Image screen, refactoring

Video chapters: Welcome Screen, View Image Screen, Refactoring.

Students now apply every earlier lesson. They build two screens, then clean up the code. Provide a background photo, a logo and a product photo in an `assets` folder.

Suggested folder structure

```
MyApp/
  App.js
  app/
    assets/        background.jpg, logo.png, chair.jpg
    components/    AppButton.js
    config/        colors.js
    screens/       WelcomeScreen.js, ViewImageScreen.js
```

Step 1: Welcome screen. Use `ImageBackground` for the full-screen photo, absolute positioning for the logo block, and two full-width buttons at the bottom.

```
// app/screens/WelcomeScreen.js
import { Image, ImageBackground, StyleSheet, Text, View } from 'react-
native';

export default function WelcomeScreen() {
  return (
    <ImageBackground style={styles.background} source=
{require('../assets/background.jpg')}>
      <View style={styles.logoContainer}>
        <Image style={styles.logo} source={require('../assets/logo.png')} />
        <Text style={styles.tagline}>Welcome to MyApp</Text>
      </View>
      <View style={styles.loginButton} />
```

```
      <View style={styles.registerButton} />
    </ImageBackground>
  );
}

const styles = StyleSheet.create({
  background: { flex: 1, justifyContent: 'flex-end', alignItems: 'center' },
  logoContainer: { position: 'absolute', top: 70, alignItems: 'center' },
  logo: { width: 100, height: 100 },
  tagline: { fontSize: 20, fontWeight: '600', paddingVertical: 20, color:
'white' },
  loginButton: { width: '100%', height: 70, backgroundColor: 'tomato' },
  registerButton: { width: '100%', height: 70, backgroundColor: 'teal' },
});
```

Step 2: View Image screen. A black screen with the photo centered and two icons pinned to the top corners. Icons come from `@expo/vector-icons`, which is included with Expo.

```
// app/screens/ViewImageScreen.js
import { Image, StyleSheet, View } from 'react-native';
import { MaterialCommunityIcons } from '@expo/vector-icons';

export default function ViewImageScreen() {
  return (
    <View style={styles.container}>
      <View style={styles.closeIcon}>
        <MaterialCommunityIcons name="close" color="white" size={35} />
      </View>
      <View style={styles.deleteIcon}>
        <MaterialCommunityIcons name="trash-can-outline" color="white" size=
{35} />
      </View>
      <Image resizeMode="contain" style={styles.image} source=
{require('../assets/chair.jpg')} />
    </View>
  );
}

const styles = StyleSheet.create({
  container: { flex: 1, backgroundColor: 'black' },
```

```
  closeIcon: { position: 'absolute', top: 40, left: 30 },
  deleteIcon: { position: 'absolute', top: 40, right: 30 },
  image: { width: '100%', height: '100%' },
});
```

Step 3: Refactor. Both screens repeat colors and the Welcome screen repeats a button. Pull the repeated pieces out:

```
// app/config/colors.js
export default { primary: 'tomato', secondary: 'teal', white: 'white', black:
'black' };
```

```
// app/components/AppButton.js
import { StyleSheet, Text, TouchableOpacity } from 'react-native';
import colors from '../config/colors';

export default function AppButton({ title, onPress, color = 'primary' }) {
  return (
    <TouchableOpacity style={[styles.button, { backgroundColor: colors[color]
}]} onPress={onPress}>
      <Text style={styles.text}>{title}</Text>
    </TouchableOpacity>
  );
}

const styles = StyleSheet.create({
  button: { width: '100%', height: 70, justifyContent: 'center', alignItems:
'center' },
  text: { color: colors.white, fontSize: 18, fontWeight: 'bold',
textTransform: 'uppercase' },
});
```

In `WelcomeScreen`, replace the two placeholder Views with `<AppButton title="Login" onPress={() => console.log('Login')} />` and `<AppButton title="Register" color="secondary" onPress={() => console.log('Register')} />`.

To see a screen, render it from `App.js`: `export default function App() { return <WelcomeScreen />; }`.

Check yourself

- Which layout technique pins the close icon to the corner, and why not Flexbox?
- What two things became easier to change after the refactor?

Activities and exercises

Four activities build on each other, ending in a graded performance task.

#	After lesson	Task	Output
1	1–2	Create an Expo project, run it on a phone or emulator, change the text, and log a message to the terminal	Screenshot of the running app and the terminal log
2	3	Build a profile card with a `View`, an `Image`, a name `Text` and a `Button` that opens an `Alert` with Yes and No options	Project folder
3	4–5	Recreate three given Flexbox layouts using colored boxes: a centered row, a column with space between, and a wrapped grid	Three screenshots plus code
4	6	Performance task: build a two-screen app (Welcome and View Image) on a theme of your choice, such as a campus or a small shop, with at least one reusable component	Working app, short demo, code

Activity 4 requirements

- Uses `ImageBackground` or `Image`, `Text`, a Touchable or `Button`, and `StyleSheet`.
- Uses Flexbox for the main layout and absolute positioning at least once.
- Contains one reusable component and one shared colors file.
- Runs on a phone or emulator without errors.

Scoring rubric for Activity 4 (100 points)

Criterion	Points	Full marks when
App runs	25	Both screens render on a device or emulator with no errors

Criterion	Points	Full marks when
Core components	20	Required components are used correctly
Layout and styling	25	Layout uses Flexbox and positioning well and looks tidy on screen
Code organization	20	Reusable component, shared colors, clear folder structure
Demo	10	Student explains what each part does in under 3 minutes

Quiz

Suggested time: 25 minutes. Part A is worth 1 point each (10 points); Part B is worth 3 points each (9 points).

Part A: Multiple choice

1. A React Native app displays its interface as:
   - a) A web page inside a WebView
   - b) Native UI elements
   - c) A streamed video
   - d) Plain HTML files
2. Expo Go lets you:
   - a) Publish to the App Store without an account
   - b) Write native Swift code
   - c) Open your running project on a phone by scanning a QR code
   - d) Replace Node.js
3. Where does `console.log` output appear in an Expo project?
   - a) In the terminal running Expo
   - b) On the phone screen
   - c) In the App Store console
   - d) In `package.json`
4. In React Native, all visible text must be placed inside which component?
   - a) `View`

- b) `Image`
- c) `Button`
- d) `Text`

5. A network image (`source={{ uri }}`) needs:
   - a) Nothing extra
   - b) An explicit width and height
   - c) Only a background color
   - d) A `Touchable` wrapper
6. Which style property name is valid in React Native?
   - a) `background-color`
   - b) `font_size`
   - c) `backgroundColor`
   - d) `bg`
7. The default `flexDirection` in React Native is:
   - a) `column`
   - b) `row`
   - c) `row-reverse`
   - d) `wrap`
8. In a default column layout, which property centers children horizontally?
   - a) `justifyContent`
   - b) `flexWrap`
   - c) `flexBasis`
   - d) `alignItems`
9. Which setting removes an element from the normal layout flow and places it against its parent's edges?
   - a) `position: 'relative'`
   - b) `position: 'absolute'`
   - c) `flexGrow: 1`
   - d) `alignSelf: 'center'`
10. Which API lets you apply different styles on iOS and Android?
    - a) `Dimensions`
    - b) `Alert`

- c) `Platform`
- d) `Image`

Part B: Short answer

11. Explain why `useWindowDimensions` is better than `Dimensions.get('window')` for handling device rotation.
12. Give two benefits of moving a repeated button into a reusable `AppButton` component.
13. Write the style object for a container that places three children in a row with equal space between them, vertically centered.

Answer key

Part A

Q	1	2	3	4	5	6	7	8	9	10
Answer	b	c	a	d	b	c	a	d	b	c

Part B: model answers (3 points each)

11. `useWindowDimensions` updates its values and re-renders the component whenever the screen size changes, including on rotation. `Dimensions.get('window')` returns the size only at the moment it is called, so the layout goes stale after rotation. Award 1 point for naming the hook, 2 for explaining the update behavior.
12. Any two of: one place to change the button's look; less repeated code; consistent appearance across screens; easier to test and reuse in other screens. Award 1.5 points for each benefit explained.
13. Model answer (award 1 point each for `flexDirection: 'row'`, `justifyContent: 'space-between'`, `alignItems: 'center'`):

```
{ flex: 1, flexDirection: 'row', justifyContent: 'space-between', alignItems: 'center' }
```

`space-around` or `space-evenly` earn full marks if the student explains that they also create equal spacing.

References and further reading

- React Native Tutorial for Beginners – Build a React Native App, Programming with Mosh (source video)
- Class Central listing and chapter list for the video, used to structure the lessons
- Get Started with React Native, official documentation
- Create a project, Expo documentation

For the next step after this module, the video's channel also offers a full React Native course for a deeper dive.